

Лабораторна робота №2

Тема: Керування середовищами. Підключення бази даних. Створення API ендпоінтів

Мета роботи: навчитися керувати середовищами, підключати бази даних, створювати API ендпоінти.

Практичні завдання

Завдання 1: Змінні середовища

- Конфігурація: У корені проєкту створено файл `.env.local`, у якому визначено змінні для різних рівнів доступу:
 - SERVER_SECRET_KEY — приватна змінна (доступна лише на сервері);
 - NEXT_PUBLIC_API_URL та NEXT_PUBLIC_ENVIRONMENT — публічні змінні (доступні і серверу, і браузеру завдяки префіксу NEXT_PUBLIC_).
- Безпека та ізоляція: Шляхом виведення даних у консоль браузера (F12) підтверджено, що секретні ключі сервера повертають undefined на стороні клієнта, що гарантує захист конфіденційної інформації від витоку.
- Логування:
 - Серверна частина: Значення всіх змінних успішно виведено в термінал Node.js через системні логи сервера.
 - Клієнтська частина: Публічні змінні виведено в консоль розробника та безпосередньо на інтерфейс сторінки /test-env.
- Результат: Перевірено коректність зчитування змінних середовища на різних етапах життєвого циклу додатка, що є базовим етапом для подальшого підключення бази даних.

Нижче продемонстровано на рис.1, 2, 3 то, як поведуть себе змінні в різних середовищах, термінал сервера, консоль браузера та на сторінці тестування.

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.121.8.000 – Лр.2		
Змн.	Арк.	№ докум.	Підпис	Дата	Звіт з лабораторної роботи №2		
Розроб.		Маковський М.С					
Перевір.		Фант М.О					
Реценз.							
Н. Контр.							
Зав.каф.							
					Літ.	Арк.	Акрушів
						1	7
					ФІКТ, гр. ВТ-23-2		

```

24 | <p><strong>NEXT_PUBLIC_API_URL:</strong> <span className="text-blue-600">{apiUrl}</span></p>
25 | <p><strong>NEXT_PUBLIC_ENVIRONMENT:</strong> <span className="text-green-600">{envMode}</span></p>
> 26 | <p><strong>SERVER_SECRET_KEY:</strong> <span className="text-red-500">{serverSecret || "HIDDEN (Server Only)"}</span></p>
27 |
28 | </div>
29 | <p className="mt-6 text-xs font-bold text-gray-400 uppercase italic">

```

Рисунок 1 – Вивід змінних у терміналі сервера.

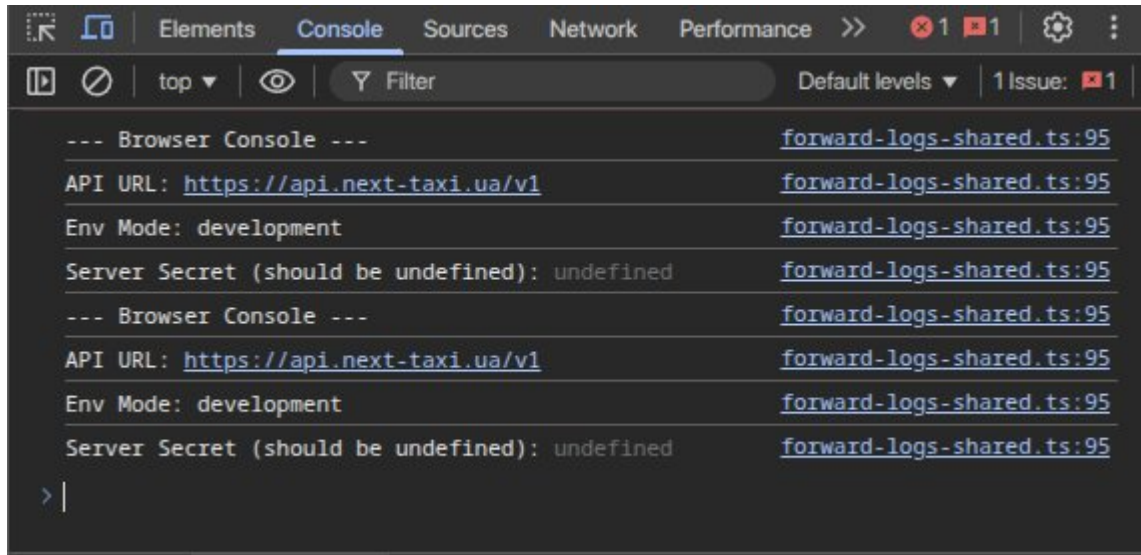


Рисунок 2 – Перевірка доступності змінних у консолі браузера.



Рисунок 3 – Відображення конфігураційних даних на сторінці тестування.

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.121.8.000 –Лр.2	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		2

Завдання 2: Створіть prod базу даних

Для реалізації системи збереження даних було обрано реляційну базу даних PostgreSQL, розгорнуту на локальному сервері. Взаємодія з базою здійснюється через сучасну ORM Prisma, для якої було спроектовано схему даних з моделлю Article. Конфігурація підключення, включаючи авторизацію користувача admin з правами CREATEDB, була безпечно винесена у файл змінних середовища .env. Після успішної синхронізації схеми за допомогою міграцій було розроблено та впроваджено TypeScript-скрипт prisma/seed.ts. Цей механізм дозволяє автоматично очищати таблиці та наповнювати їх початковими тестовими даними, що було верифіковано через консольні логи терміналу та графічний інтерфейс Prisma Studio(рис.4).

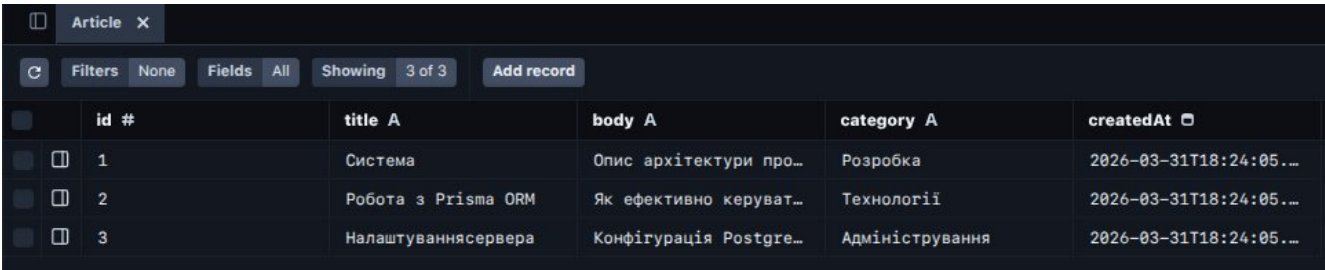


Рисунок 4 – Графічний інтерфейс Prisma Studio.

Завдання 3: Створіть dev базу даних

Для створення ізольованого середовища розробки було використано технологію контейнеризації Docker. За допомогою офіційного образу PostgreSQL розгорнуто окремий контейнер, що працює на альтернативному порті 5434, аби уникнути конфліктів із основною базою даних. Керування підключеннями реалізовано через змінні середовища у файлі .env, що дозволяє гнучко перемикатися між середовищами шляхом зміни параметрів порту та назви бази даних. Після успішного запуску контейнера(рис.5) було застосовано міграції Prisma та виконано скрипт seed.ts, що забезпечило ідентичність структур даних та наявність тестового контенту в dev-середовищі, аналогічно до основної бази.

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
522d0115a1e9	postgres	"docker-entrypoint.s..."	38 seconds ago	Up 37 seconds	0.0.0.0:5434->5432/tcp, [::]:5434->5432/tcp	fullstack-dev

Рисунок 5 – Запуск контейнера docker.

Завдання 4: Створіть API ендпоінти

Для забезпечення взаємодії між клієнтською частиною додатка та базою даних було реалізовано програмний інтерфейс CRUD API на основі механізму Route Handlers у Next.js. У межах ендпоінту `/api/articles` (рис. 8) розроблено обробники для чотирьох основних HTTP-методів: GET для вибірки статей, POST для створення нових записів, PUT для модифікації та DELETE для видалення об'єктів. Взаємодія з базою даних на рівні бекенду реалізована через Prisma Client. На фронтенд-частині сторінку статей було переведено на використання власного API за допомогою асинхронних запитів `fetch` (рис. 9). Працездатність розроблених методів була підтверджена комплексним тестуванням: через вкладку Network у браузері (рис. 10) для контролю запитів з фронтенду та за допомогою інструментів командного рядка (рис. 11) для перевірки незалежної роботи API у форматі JSON.

```
src > app > api > articles > routes.ts > ...
1  import { NextResponse } from 'next/server';
2  import { PrismaClient } from '@prisma/client';
3
4  const prisma = new PrismaClient();
5
6
7  export async function GET() {
8    const articles = await prisma.article.findMany();
9    return NextResponse.json(articles);
10 }
11
12
13 export async function POST(request: Request) {
14   const body = await request.json();
15   const newArticle = await prisma.article.create({
16     data: {
17       title: body.title,
18       body: body.body,
19       category: body.category,
20     },
21   });
22   return NextResponse.json(newArticle, { status: 201 });
23 }
24
25
26 export async function PUT(request: Request) {
27   const body = await request.json();
28   const updatedArticle = await prisma.article.update({
29     where: { id: body.id },
30     data: { title: body.title, body: body.body },
31   });
32   return NextResponse.json(updatedArticle);
33 }
34
35
36 export async function DELETE(request: Request) {
37   const { id } = await request.json();
38   await prisma.article.delete({
39     where: { id: Number(id) },
40   });
41   return NextResponse.json({ message: 'Статтю видалено' });
42 }
```

Рисунок 8 – Реалізація методів GET, POST, PUT, DELETE у файлі route.ts.

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.121.8.000 –Лр.2	Арк.
						4
Змн.	Арк.	№ докум.	Підпис	Дата		

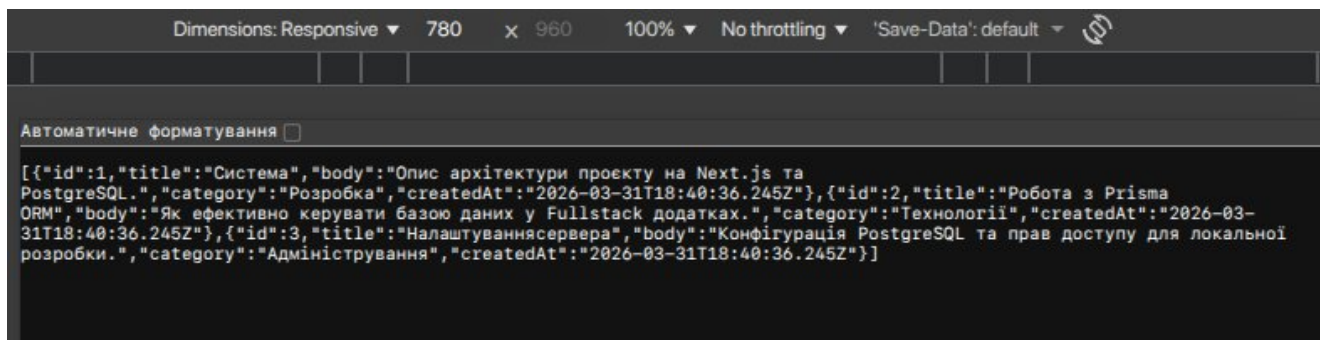


Рисунок 9 – Відображення отриманих із бази даних статей на фронтенд-сторінці додатка.

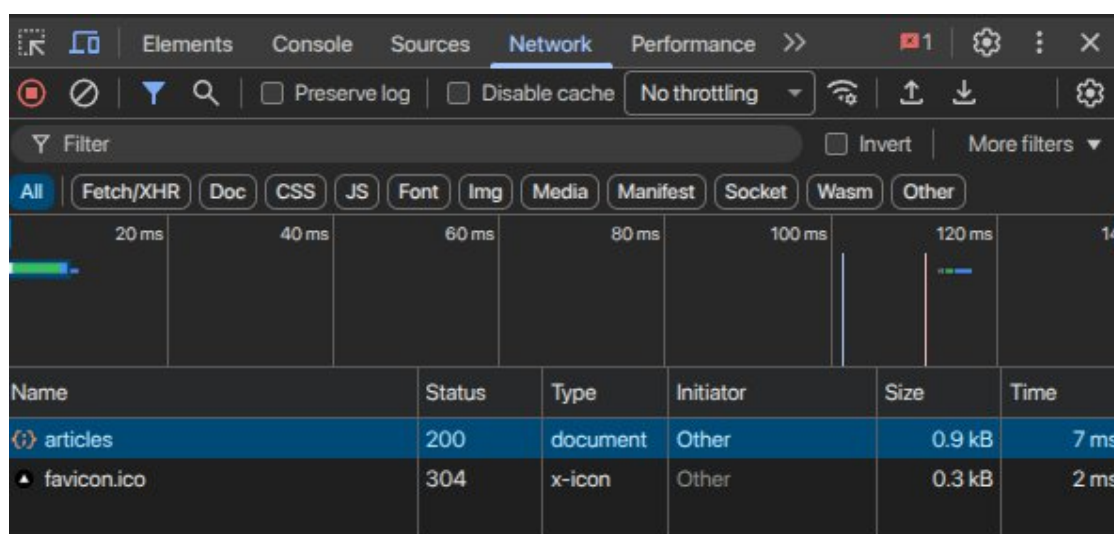


Рисунок 10 – Аналіз мережевої активності у вкладці Network при зверненні до API.

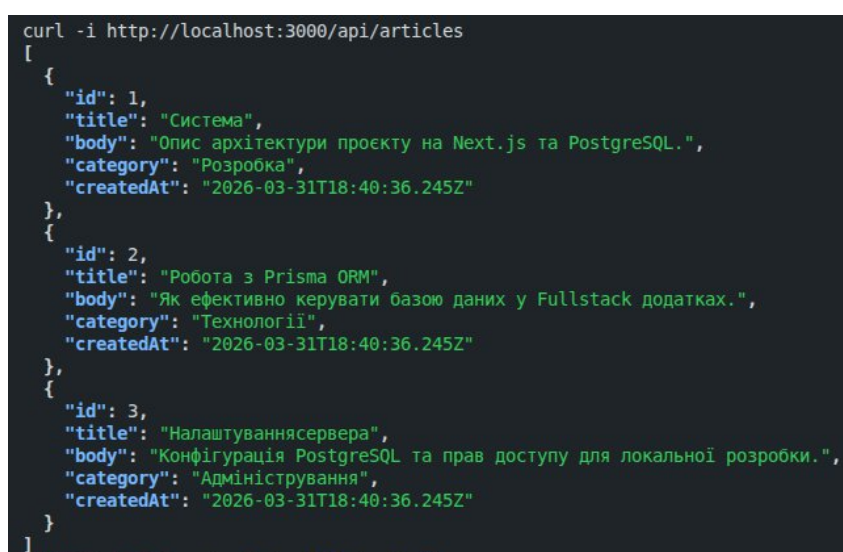


Рисунок 11 – Тестування API ендпоінту через термінал за допомогою curl.

Завдання 5: Створіть клієнтські сторінки, які використовують ендпоінти з завдання 4

Для реалізації динамічного відображення даних на стороні клієнта було впроваджено бібліотеку SWR (Stale-While-Revalidate), що дозволило оптимізувати роботу з API та забезпечити автоматичне оновлення контенту без повного перезавантаження сторінки. У програмному коді клієнтського компонента (рис. 12) було налаштовано хук useSWR, який звертається до розробленого ендпоінту /api/articles та використовує функцію-завантажувач fetcher для отримання JSON-даних. Такий підхід забезпечив ефективне кешування та швидкий відгук інтерфейсу. Візуальне оформлення сторінки було виконано згідно з дизайн-концепцією проекту Taxi-Town (рис. 13), розробленою в межах ЛР1: використано фірмову чорно-жовту палітру, акцентну типографіку та стилістику бруталізму з масивними тінями. Сторінка адаптована для зручного перегляду списку статей, має логічну структуру та чіткі заклики до дії (CTA), що підтверджує успішну інтеграцію фронтенд-частини додатка з серверною базою даних PostgreSQL через проміжний шар API.

```
1  // app > (next) > articles > page.tsx > ArticlesPage
2  "use client";
3  import Link from "next/link";
4  import useSWR from "swr";
5
6  // Функція-завантажувач для SWR
7  const fetcher = (url: string) => fetch(url).then((res) => res.json());
8
9  export default function ArticlesPage() {
10   const { data: articles, error, isLoading } = useSWR("/api/articles", fetcher, {
11     refreshInterval: 5000, // Автоматичне оновлення кожні 5 секунд
12   });
13
14   if (error) return <div className="p-6 text-red-500 font-bold">Помилка завантаження даних...</div>;
15   if (isLoading) return <div className="p-6 text-taxi-yellow font-black animate-pulse">ЗАВАНТАЖЕННЯ ДАНИХ ТАКШІ...</div>;
16
17   return (
18     <div className="py-6 px-2 max-w-5xl mx-auto">
19       <h1 className="text-4xl font-black text-taxi-black uppercase mb-8 border-b-4 border-taxi-yellow inline-block italic">
20         База даних
21       </h1>
22       <div className="text-taxi-yellow bg-taxi-black px-2 mt-2">Taxi-DB</div>
23       {articles ? articles.length === 0 ? (
24         <p className="text-taxi-gray font-bold uppercase tracking-tighter">
25           База порожня. Пореєд Docker та запустити Seed.
26         </p>
27       ) : (
28         <div className="grid grid-cols-1 laptop:grid-cols-2 gap-6">
29           {articles.map((art: any) => (
30             <div key={art.id} href={`/articles/${art.id}`} className="group flex items-center bg-white border-2 border-taxi-black p-5 shadow-[6px_6px_0px_rgba(0,0,0,1)] hover:shadow-none hover:transf
31               <span className="text-3xl font-black text-taxi-yellow mr-5 group-hover:scale-110 transition-transform">
32                 {art.id.toString().padStart(2, '0')}
33               </span>
34               <div className="flex flex-col truncate">
35                 <span className="text-taxi-black font-black uppercase text-lg group-hover:text-taxi-yellow transition-colors truncate">
36                   {art.title}
37                 </span>
38                 <span className="text-[18px] font-bold text-white bg-taxi-black px-2 py-0.5 w-fit uppercase tracking-widest mt-1">
39                   {art.category}
40                 </span>
41               </div>
42               <span className="ml-auto text-2xl font-black text-taxi-black group-hover:translate-x-2 transition-transform">
43                 =
44               </span>
45             </div>
46           ))}
47         </div>
48       )
49     </div>
50   );
51 }
```

Рисунок 12 – Реалізація клієнтського запиту з використанням бібліотеки SWR у файлі page.tsx.

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.121.8.000 –Лр.2	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		6

КОНТЕНТ-МЕНЕДЖЕР



ОБРАНЕ

+ СТВОРИТИ

ВСІ СТАТТІ TAXI-DB

01 СИСТЕМА



02 РОБОТА З PRISMA ORM



03 НАЛАШТУВАННЯ СЕРВЕРА



Рисунок 13 – Фінальний вигляд сторінки статей у фірмовому стилі проекту.

Висновок: У ході виконання лабораторної роботи було розроблено повноцінний Fullstack-додаток на базі фреймворку Next.js, що охоплює всі етапи створення сучасної веб-системи: від розгортання реляційної бази даних PostgreSQL у контейнері Docker та налаштування ORM Prisma для керування схемами й міграціями, до реалізації серверних CRUD-ендпоінтів та інтеграції клієнтської логіки за допомогою бібліотеки SWR. Практична реалізація підтвердила ефективність обраного технологічного стеку для забезпечення швидкої взаємодії між фронтендом та бекендом, а стилізація інтерфейсу в межах концепції Taxi-Town дозволила поєднати складну технічну архітектуру з адаптивним та зручним дизайном користувача. Отримані результати демонструють повну готовність системи до масштабування та стабільної роботи в ізольованому середовищі розробки.